

k-Nearest Neighbor (KNN) และการประยุกต์ใช้งาน

การจำแนกประเภท (Classification) และการจัดกลุ่ม (Clustering) จากชุดข้อมูล COVID-19
ระดับเทศมณฑลของสหรัฐอเมริกา

รายงานฉบับนี้นำเสนอการศึกษาอัลกอริทึม k-Nearest Neighbor (KNN) ที่พัฒนาด้วย TensorFlow โดยประยุกต์ใช้กับชุดข้อมูล COVID-19 ระดับเทศมณฑล (county) ของสหรัฐอเมริกา จาก The New York Times ครอบคลุมการประยุกต์ใช้งาน 2 ด้าน ได้แก่ (1) การจำแนกประเภท (Classification) ด้วย KNN แบบมีผู้สอน (Supervised) เพื่อทำนายระดับความเสี่ยงของ COVID-19 ในแต่ละเทศมณฑล และ (2) การจัดกลุ่ม (Clustering) ด้วย K-Means โดยใช้ KNN เป็นเครื่องมือเสริมสำหรับการจัดกลุ่มข้อมูลใหม่และการให้คะแนนความเกาะกลุ่ม

โปรเจกต์: ML-4-KNN

1. บทนำ

k-Nearest Neighbor (KNN) เป็นอัลกอริทึมแบบไม่มีพารามิเตอร์ (non-parametric) ที่เรียบง่าย โดยทำนายค่าของจุดข้อมูลใหม่จากการพิจารณาจุดข้อมูลที่อยู่ใกล้ที่สุด k จุดในชุดข้อมูลอ้างอิงที่มีป้ายกำกับ (labeled) อัลกอริทึมนี้สามารถใช้ได้โดยตรงสำหรับการจำแนกประเภท (การโหวตส่วนใหญ่จากเพื่อนบ้าน) หรือใช้เป็นเครื่องมือสนับสนุนภายในไปป์ไลน์อื่น เช่น การกำหนดจุดข้อมูลใหม่เข้าสู่กลุ่มที่มีอยู่แล้ว โปรเจกต์นี้สาธิตบทบาททั้งสองแบบโดยใช้ชุดข้อมูลจริงชุดเดียวกัน

การคำนวณระยะทางทั้งหมด รวมถึงอัลกอริทึม K-Means ที่ใช้ในการจัดกลุ่ม ถูกพัฒนาขึ้นโดยตรงด้วยการดำเนินการเทนเซอร์ (tensor operations) ของ TensorFlow (การคำนวณระยะทางแบบคู่ (pairwise distance) ผ่านสูตร $\|a-b\|^2 = \|a\|^2 + \|b\|^2 - 2(a \cdot b)$ และใช้ `tf.math.top_k` สำหรับการเลือกเพื่อนบ้าน) แทนที่จะเรียกใช้ฟังก์ชันสำเร็จรูปจากไลบรารี เพื่อให้กลไกการทำงานของอัลกอริทึม มีความชัดเจนและตรวจสอบได้

2. ชุดข้อมูลและการสร้างคุณลักษณะ (Feature Engineering)

แหล่งที่มา: ชุดข้อมูล COVID-19 ระดับเทศมณฑลของสหรัฐฯ จาก The New York Times (`us-counties.csv`) ซึ่งเป็นข้อมูลจำนวนผู้ติดเชื้อและผู้เสียชีวิตสะสมรายวันของแต่ละเทศมณฑล มีประมาณ 2.5 ล้านแถว ครอบคลุมช่วงเวลา 21 มกราคม 2020 ถึง 13 พฤษภาคม 2022 (ดูรายละเอียดใน `link-data.txt`)

เนื่องจากไฟล์ข้อมูลดิบเป็นอนุกรมเวลารายวันที่มีความยาวมาก จึงมีการรวบรวมข้อมูล (aggregate) ให้เหลือ 1 แถวต่อ 1 เทศมณฑล (รวม 3,139 เทศมณฑล หลังจากคัดกรองเทศมณฑลที่มีผู้ติดเชื้อสะสมน้อยกว่า 50 ราย และแถวที่ระบุเทศมณฑลเป็น “Unknown” ออก) พร้อมสร้างคุณลักษณะ (features) ดังนี้:

คอลัมน์	ความหมาย
<code>total_cases</code>	จำนวนผู้ติดเชื้อสะสม ณ สิ้นสุดอนุกรมเวลา
<code>total_deaths</code>	จำนวนผู้เสียชีวิตสะสม ณ สิ้นสุดอนุกรมเวลา
<code>death_rate_pct</code>	$\text{total_deaths} / \text{total_cases} \times 100$
<code>duration_days</code>	จำนวนวันระหว่างวันแรกและวันสุดท้ายที่มีการบันทึกผู้ติดเชื้อ
<code>peak_daily_new_cases</code>	จำนวนผู้ติดเชื้อรายใหม่สูงสุดในหนึ่งวัน
<code>avg_daily_new_cases</code>	ค่าเฉลี่ยผู้ติดเชื้อรายใหม่ต่อวันตลอดช่วงที่มีการบันทึก
<code>std_daily_new_cases</code>	ส่วนเบี่ยงเบนมาตรฐานของผู้ติดเชื้อรายใหม่ต่อวัน (ความผันผวนของการระบาด)
<code>case_growth_rate</code>	$\text{total_cases} / \text{duration_days}$
<code>risk_level</code>	ตัวแปรเป้าหมายสำหรับการจำแนกประเภท: Low / Medium / High แบ่งตามเทอร์ไทล์ (tertile) ของ <code>death_rate_pct</code>

3. การประยุกต์ใช้งานที่ 1 — การจำแนกประเภทด้วย KNN

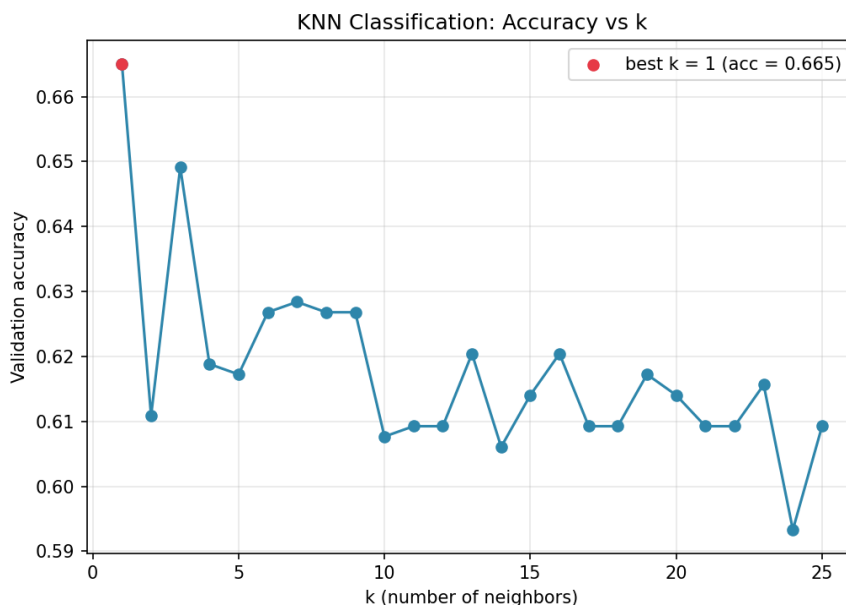
เป้าหมาย: ทำนายค่า `risk_level` (ระดับความเสี่ยง Low / Medium / High จากอัตราการเสียชีวิตของ COVID-19) ของแต่ละเทศมณฑล จากคุณลักษณะที่แสดงรูปแบบของการระบาด ได้แก่ `total_cases`, `total_deaths`, `duration_days`, `peak_daily_new_cases`, `avg_daily_new_cases`, `std_daily_new_cases` และ `case_growth_rate` โดย ไม่รวม `death_rate_pct` เป็นคุณลักษณะนำเข้า เนื่องจากตัวแปรเป้าหมายถูกสร้างขึ้นจากค่านี้โดยตรง หากนำมาใช้จะทำให้เกิดการรั่วไหลของคำตอบ (target leakage)

3.1 วิธีการ

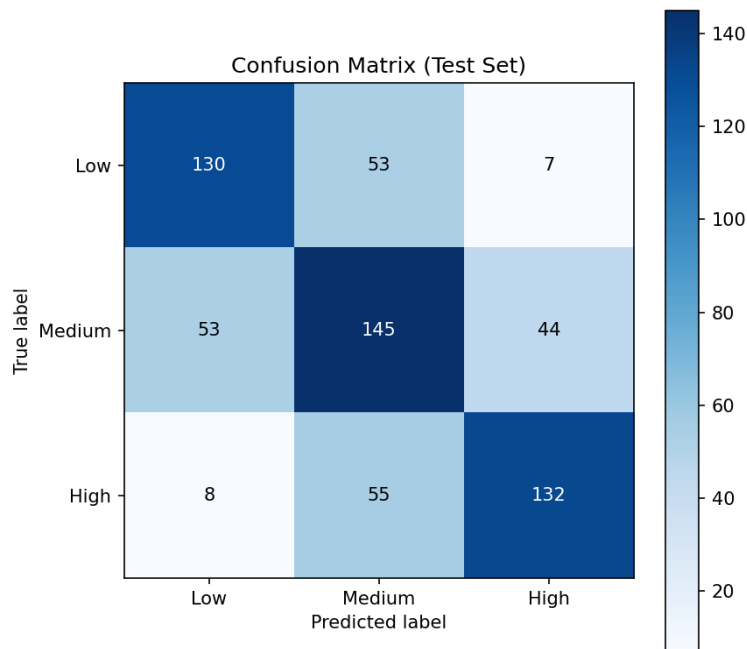
ตัวจำแนกประเภท KNN (`knn_tf.py`) คำนวณเมตริกซ์ระยะทางกำลังสองแบบคู่ (pairwise squared-distance) ระหว่างจุดข้อมูลที่ต้องการทำนายกับชุดข้อมูลฝึกฝนทั้งหมดในการดำเนินการเวกเตอร์ครั้งเดียวด้วย TensorFlow จากนั้นเลือกระยะทางที่น้อยที่สุด `k` อันดับด้วย `tf.math.top_k` และทำนายป้ายกำกับ (label) ตามคะแนนเสียงส่วนใหญ่ของเพื่อนบ้านทั้ง `k` ตัว โดยใช้ one-hot encoding และการรวมผลรวม

ข้อมูลเทศมณฑลถูกแบ่งเป็น 60% สำหรับฝึกฝน (train), 20% สำหรับตรวจสอบ (validation) และ 20% สำหรับทดสอบ (test) คุณลักษณะทั้งหมดถูกปรับมาตรฐาน (z-score) โดยใช้สถิติจากชุดฝึกฝนเท่านั้น จำนวนเพื่อนบ้าน `k` ถูกทดลองตั้งแต่ 1 ถึง 25 และเลือกค่าที่ดีที่สุดโดยใช้ชุด ตรวจสอบ (validation) เท่านั้น ส่วนชุดทดสอบ (test) จะถูกประเมินเพียงครั้งเดียวหลังจากกำหนดค่า `k` แล้ว เพื่อหลีกเลี่ยงการปรับแต่งโมเดลด้วยข้อมูลชุดเดียวกับที่ใช้รายงานความแม่นยำสุดท้าย

3.2 ผลลัพธ์



ภาพที่ 1 ความแม่นยำบนชุดตรวจสอบ (validation) สำหรับ `k = 1..25` โดยเลือกค่า `k` ที่ดีที่สุดจากชุดตรวจสอบเท่านั้น



ภาพที่ 2 เมทริกซ์ความสับสน (Confusion Matrix) บนชุดทดสอบที่แยกไว้ต่างหาก ณ ค่า k ที่ดีที่สุด

ผลการทดลองบนชุดตรวจสอบเลือกค่า $k = 1$ ให้ความแม่นยำบนชุดทดสอบประมาณ 0.65 ค่า F1 ของแต่ละประเภทอยู่ที่ประมาณ 0.68 (Low), 0.59 (Medium) และ 0.70 (High) แสดงให้เห็นว่าโมเดลสามารถแยกแยะคุณสมบัติที่มีความเสี่ยงสุดขีด (Low และ High) ได้ดีกว่ากลุ่มความเสี่ยงระดับกลาง (Medium) อย่างชัดเจน ซึ่งเป็นผลที่คาดการณ์ได้

เนื่องจากขอบเขตของประเภทที่แบ่งตามเทอร์โทลล์ทำให้ประเภทกลางแยกแยะจากประเภท ข้างเคียงทั้งสองด้านได้ยากกว่า ผลการทำนายรายคุณสมบัติทั้งหมดถูกบันทึกไว้ในไฟล์ `classification/outputs/predictions.csv`

4. การประยุกต์ใช้งานที่ 2 — การจัดกลุ่มด้วย K-Means (โดยใช้ KNN เป็นเครื่องมือเสริม)

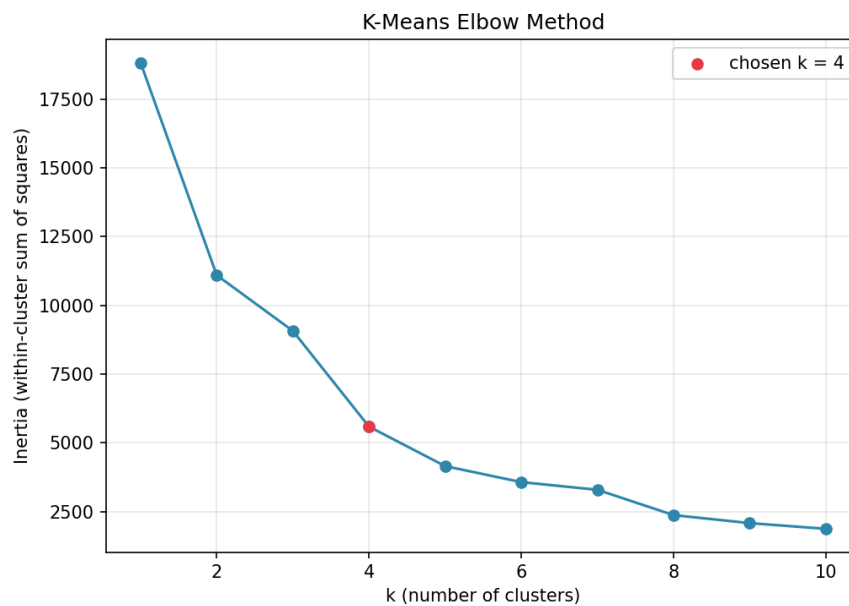
เป้าหมาย: จัดกลุ่มเทศมณฑลที่มีรูปแบบการระบาดโดยรวม (ขนาด ความรุนแรง และรูปแบบการเติบโต) ใกล้เคียงกัน โดยใช้คุณลักษณะ: total_cases, total_deaths, death_rate_pct, peak_daily_new_cases, avg_daily_new_cases และ case_growth_rate เนื่องจากการจัดกลุ่มเป็นการเรียนรู้แบบไม่มีผู้สอน (unsupervised) จึงสามารถรวม death_rate_pct เข้าไว้เป็นคุณลักษณะได้โดยไม่มีปัญหาการรั่วไหลของคำตอบ

4.1 วิธีการ

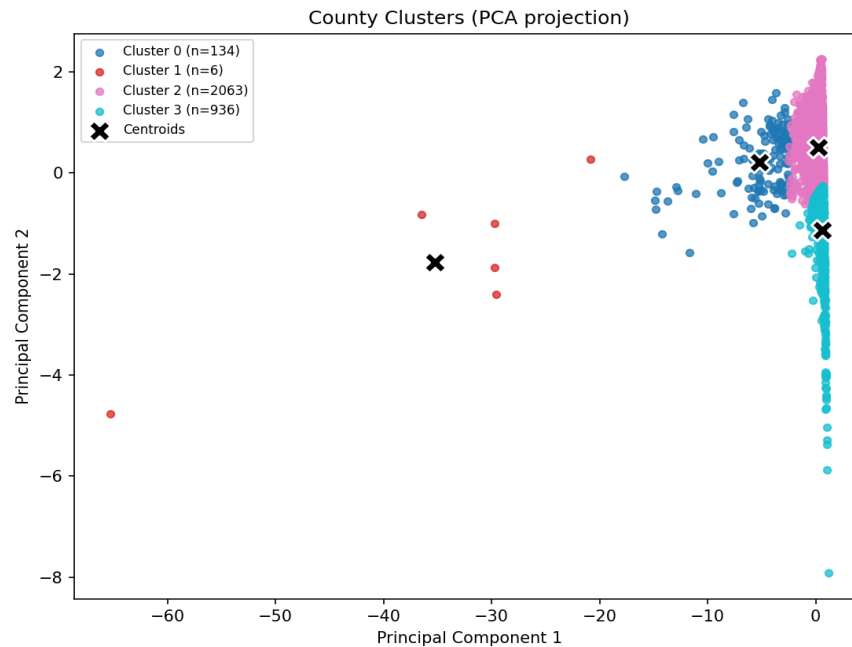
K-Means (kmeans_tf.py) พัฒนาขึ้นโดยใช้อัลกอริทึมของ Lloyd โดยตรงด้วย TensorFlow พร้อมการกำหนดจุดเริ่มต้นแบบ k-means++ เพื่อความเสถียรในการลู่เข้า และใช้ฟังก์ชันคำนวณระยะทางแบบคู่แบบเวกเตอร์เดียวกับตัวจำแนกประเภทส่วน KNN ถูกนำมาใช้เป็นเครื่องมือเสริม 2 บทบาท (knn_tools.py) ได้แก่:

- label_new_points() — กำหนดกลุ่มให้กับเทศมณฑลใหม่/ที่ยังไม่เคยเห็น โดยใช้การโหวตส่วนใหญ่แบบ KNN จากเทศมณฑลที่จัดกลุ่มไว้แล้ว ซึ่งเป็นอีกแนวทางหนึ่งนอกเหนือจากการกำหนดกลุ่มตามศูนย์กลาง (centroid) ที่ใกล้ที่สุด
- cluster_cohesion() — คำนวณระยะทางเฉลี่ยของแต่ละเทศมณฑลไปยังเพื่อนบ้าน k ตัวที่ใกล้ที่สุดภายในกลุ่มเดียวกัน เพื่อใช้เป็นคะแนนความหนาแน่น/ความเกาะกลุ่มอย่างง่ายของแต่ละเทศมณฑล

4.2 ผลลัพธ์



ภาพที่ 3 วิธี Elbow: ค่าความคลาดเคลื่อนภายในกลุ่ม (inertia) เทียบกับค่า k โดยเลือก k = 4 เนื่องจากผลตอบแทนที่ลดลงหลังจากจุดนี้



ภาพที่ 4 ผลลัพธ์การจัดกลุ่มทั้ง 4 กลุ่ม แสดงในรูปแบบ 2 มิติด้วยเทคนิค PCA

กลุ่ม	จำนวนเทศมณฑล	ผู้ติดเชื้อสะสมเฉลี่ย	อัตราการเสียชีวิตเฉลี่ย (%)	ลักษณะเด่น
0	134	~223,700	1.13%	เทศมณฑลเขตเมืองใหญ่ ปริมาณผู้ติดเชื้อสูง
1	6	~1,408,800	1.05%	กลุ่มค่าผิดปกติสุดขั้ว — เทศมณฑลที่ใหญ่ที่สุดในสหรัฐฯ (Los Angeles, Maricopa, Miami-Dade, Cook, Harris, San Diego)
2	2,063	~16,800	1.20%	เทศมณฑลขนาดกลาง/เล็กทั่วไป — คิดเป็นส่วนใหญ่ของประเทศ
3	936	~6,100	2.35%	เทศมณฑลขนาดเล็กที่มีอัตราการเสียชีวิตสูงอย่างไม่สมส่วน

จากกราฟ Elbow พบว่าค่าความคลาดเคลื่อนเริ่มลดลงอย่างช้าลงอย่างชัดเจนหลังจาก $k = 4$

จึงเลือกใช้จำนวนกลุ่มสุดท้ายเท่ากับ 4 กลุ่มที่ 1 แยกเทศมณฑลที่มีผู้ติดเชื้อสูงสุด 6 อันดับแรกของสหรัฐฯ

ออกมาอย่างชัดเจนในฐานะค่าผิดปกติ ส่วนกลุ่มที่ 3 มีความน่าสนใจตรงที่มีอัตราการเสียชีวิตเฉลี่ยสูงกว่าอย่างมีนัยสำคัญ (2.35%) แม้จะมีจำนวนผู้ติดเชื้อเฉลี่ย น้อยกว่ากลุ่มที่ 0 และกลุ่มที่ 2 ก็ตาม

ซึ่งบ่งชี้ว่าเทศมณฑลขนาดเล็กเหล่านี้ได้รับผลกระทบรุนแรงอย่างไม่สมส่วนเมื่อเทียบกับ ขนาดของการระบาด

จากการทดสอบกับกลุ่มตัวอย่าง 20 เทศมณฑล พบว่าการกำหนดกลุ่มด้วยวิธีโหวตแบบ KNN ให้ผลตรงกับการกำหนดกลุ่มโดยตรงจาก K-Means ถึง 100% ซึ่งยืนยันว่าทั้งสองแนวทางมีความสอดคล้องกัน

5. บทสรุป

โปรเจกต์นี้แสดงให้เห็นการประยุกต์ใช้ KNN ในสองบทบาทที่เสริมกันบนชุดข้อมูลจริงชุดเดียวกัน ได้แก่ การเป็นตัวจำแนกประเภทแบบมีผู้สอนโดยตรงสำหรับระดับความเสี่ยงของเทศกาล (ความแม่นยำบนชุดทดสอบประมาณ 65% จาก 3 ประเภท) และการเป็นเครื่องมือสนับสนุนการจัดกลุ่มด้วย K-Means — ทั้งในการกำหนดกลุ่มให้ชุดข้อมูลใหม่ และการให้คะแนนความเกาะกลุ่มรายจุด การพัฒนาการคำนวณระยะทางหลักโดยตรงด้วย TensorFlow ช่วยให้กลไกการทำงาน ของอัลกอริทึมมีความโปร่งใส โดยการทำนายทุกครั้งสามารถสืบทอดกลับไปยังเมตริกระยะทางแบบคู่และการโหวตของเพื่อนบ้าน k อันดับได้อย่างชัดเจน แทนที่จะเป็นการเรียกใช้ไลบรารีแบบกล่องดำ

ไฟล์ในโปรเจกต์: dataset.csv (ชุดข้อมูลเทศกาลแบบแบบ), classification/ และ clustering/ (ไปป์ไลน์และผลลัพธ์ทั้งหมด), requirements.txt และ link-data.txt (แหล่งที่มาของข้อมูลดิบ)